

Industrialisation et Déploiement MLOps

Comment passer d'un projet local à un environnement de production sécurisé

1. Le Projet et le Besoin Traité

Le Contexte :

- Une application Data/IA fonctionnelle en local (Django + FastAPI + PostgreSQL).
- Objectif : Rendre cette application accessible publiquement de manière fiable pour des démonstrations et faciliter les futures mises à jour.

Les Défis de l'Industrialisation :

- Sécuriser un VPS OVH vierge exposé sur Internet.
- Ne pas exposer les conteneurs applicatifs directement.
- Automatiser la livraison (CI/CD) pour éviter les erreurs humaines.
- Gérer les secrets (mots de passe, clés d'API) de manière sécurisée.

2. Les Mesures de Sécurité Appliquées au VPS

Un serveur nu est une cible. Voici la forteresse mise en place :

- **Mise à jour système** : `apt update && apt upgrade` à la première connexion.
- **Utilisateur dédié (amaury)** : Fin de l'utilisation de `root` pour les tâches courantes.
- **Accès SSH par clé Ed25519** : Désactivation totale de l'authentification par mot de passe.
- **Changement du port SSH** : Le port 22 a été fermé et remplacé par le port 1455 pour éviter les scripts automatisés de force brute.
- **Pare-feu (UFW)** : Seuls les ports vitaux sont ouverts (80 HTTP, 443 HTTPS, 1455 SSH).
- **Fail2ban** : Bannissement automatique des adresses IP tentant de forcer l'accès.

3. L'Architecture de Déploiement

Pourquoi utiliser ces technologies ?

- **VPS** : L'hôte physique qui exécute notre application.
- **Docker & Docker Compose** : Garantissent que l'application tournera sur le serveur *exactement* comme elle tournait en local (reproductibilité).
- **Traefik (Reverse Proxy)** : Le gardien de la porte. C'est le seul composant exposé sur les ports 80 et 443. Il route les requêtes vers Django ou FastAPI sans exposer leurs ports internes.
- **Let's Encrypt** : Intégré à Traefik, il génère le certificat HTTPS automatiquement pour protéger les données en transit.



4. La Gestion des Secrets : Zéro `.env` sur le VPS

Le Risque : Exposer des secrets (mots de passe BDD, clé Django) dans le code source ou dans un fichier en clair sur le serveur.

La Solution CI/CD :

1. Les secrets sont stockés de manière cryptée dans **GitHub Secrets**.
2. GitHub Actions se connecte au VPS via SSH.
3. Les secrets sont **injectés en tant que variables d'environnement en mémoire vive** directement dans la commande `docker compose up`.
4. **Bilan :** Aucun fichier contenant des mots de passe n'existe sur le disque dur du VPS.

⚙️ 5. La Pipeline CI/CD et la Registry

```
flowchart LR
    A[Git Push] --> B[GitHub Actions]
    B -->|1. Test| C[Pytest]
    B -->|2. Build & Tag| D[Docker Build]
    D -->|3. Publish| E[(GHCR Registry)]
    E -->|4. Deploy| F[VPS : Pull & Run]
```

- **Tests automatisés** : Bloquent le déploiement si le code est cassé.
- **GitHub Container Registry (GHCR)** : Stocke chaque version de notre image Docker. Permet un retour arrière immédiat si besoin (Rollback).
- **Déploiement Automatisé** : Connexion SSH, téléchargement des images depuis GHCR, et redémarrage sans interruption (`up -d`).

6. Incidents Rencontrés & Résolution

Incident 1 : Traefik "404 Page Not Found"

- *Diagnostic* : Traefik tentait de communiquer avec l'API Docker du VPS (v29) en utilisant une version obsolète (v1.24). L'API de Docker Engine bloquait la connexion, rendant Traefik "aveugle" aux conteneurs.
- *Solution* : Forcer `DOCKER_API_VERSION=1.41` dans le Compose et mettre à jour l'image Traefik vers la v3.

Incident 2 : Let's Encrypt refuse de générer le HTTPS

- *Diagnostic* : Let's Encrypt a refusé l'email `admin@example.com` défini par défaut. Traefik a mis en cache cet échec dans `acme.json`, bloquant toute nouvelle tentative même après correction du fichier yaml.
- *Solution* : Nettoyage manuel du volume Docker (`rm -f /letsencrypt/acme.json`) via un conteneur éphémère Alpine, pour forcer Traefik à repartir de zéro.

7. Démonstration de la Release

Scénario de la démonstration :

1. Visite du site actuel (`https://docker-political.duckdns.org`).
2. Modification légère du code source en direct (ex: texte sur la page d'accueil).
3. `git commit` et `git push` .
4. Suivi visuel du pipeline GitHub Actions (Tests -> Build -> Deploy).
5. Actualisation du navigateur pour prouver la mise en production de la release.
6. Connexion rapide au VPS pour prouver l'absence de secret sur le disque.

8. Limites Actuelles et Améliorations Futures

Ce VPS est parfait pour une démonstration pédagogique, mais en **production réelle d'entreprise**, il faudrait aller plus loin :

1. **Haute Disponibilité** : Remplacer le VPS unique par un cluster (Kubernetes ou Docker Swarm) pour éviter le *Single Point Of Failure* (SPOF).
2. **Base de données Managée** : Ne pas héberger PostgreSQL dans un conteneur sur le même serveur, mais utiliser un service cloud managé avec sauvegardes automatiques.
3. **Supervision & Alerting** : Installer Prometheus et Grafana pour surveiller la RAM, le CPU et recevoir un mail si l'application tombe.
4. **CI/CD** : Mettre en place des tests de charge et un environnement de *Staging* (pré-production) avant de déployer en production.

Merci de votre écoute ! Des questions ?